import os import
random
openaction
js
javascript
dfgdfbdcxvbfbxf
gdbxfg
openaction
js
javascript
openaction
js
javascript
kujhoiygouiyhdv
df
def generate_dummy_dropper(filename="dummy_dropper_test.pdf"):
# 1. The Magic Bytes (Looks like a real PDF)
pdf_header = b"%PDF-1.5\n%\xE2\xE3\xCF\xD3\n"
# 2. The Structural Anomalies (The Triggers)
# We include the /OpenAction and /JS tags to trigger your tag scanner
malicious_tags = b"""

```
1 0 obj
<< /Type /Catalog /Pages 2 0 R /OpenAction 3 0 R >> endobj
2 0 obj
<< /Type /Pages /Kids [4 0 R] /Count 1 >> endobj
3 0 obj
<< /Type /Action /S /JavaScript /JS (app.alert("e-Kavach Test: Simulated Payload Blocked!");) >
> endobj
4 0 obj
<< /Type /Page /Parent 2 0 R /MediaBox [0 0 612 792] >> endobj
"""
# 3. The Obfuscation Trap (High Entropy Data)
# We generate 5000 bytes of pure, chaotic random data to simulate an AES-encrypted payload
# This will cause your Entropy Scanner to spike to ~7.99 high_entropy_payload
= b"\n% e-Kavach Simulated Encrypted Payload stream\n" high_entropy_payload
+= b"\n% end stream\n"
# 4. Write to file with
open(filename, "wb") as f:
f.write(pdf_header)
f.write(malicious_tags)
f.write(high_entropy_payload)
```